Data Lakes for Big Data [Report on Existing Data Lakes]

Abdelghny Orogat
abdelghny.orogat@carleton.ca
Carleton University

KEYWORDS

Data Lakes, Metadata, Open data

1 ABSTRACT

This is a report that summarizes the book *"Databases and big data set Laurent, Anne Laurent, Dominique Madera, Cedrine - Data lakes (2020, ISTE, Ltd., Wiley) - libgen.liFile"* and on existing Data Lakes and their current features and limitations.

2 INTRODUCTION

James Dixon, a Penthao CTO, coined the word data lake for the first time [4]. Dixon predicted that data lakes would be massive collections of raw data, structured or unstructured, that users could use for sampling, mining, or analytical purposes in this seminal work. The idea of a data lake, according to Gartner [2] in 2014, was nothing more than a modern way of storing data at low cost. This point was revised a few years later, based on the fact that data lakes are now considered important in many businesses [7]. As a result, Gartner now finds the data lake model to be the holy grail of knowledge management when it comes to innovating through data value.

The characteristics of data lakes are described as storing data in its original state at low cost, according to one of the earliest academic papers about data lakes [1]. Since (1) data servers are inexpensive and (2) no data transformation, cleaning, or planning is needed, the cost is kept low. In 2016, Bill Inmon published a book on a data lake architecture [3] in which the issue of storing useless or impossible to use data is addressed. More specifically, Bill Inmon argues in this book that the data lake architecture should move towards information systems, rather than storing only raw data, in order to avoid storing "prepared" data via a method like ETL (Extract-Transform-Load), which is commonly used in data warehouses.

Following other works, the most influential work on data lake architecture, modules, and positioning is discussed in IBM [6], since the focus is on data governance, specifically the metadata catalog. The authors of [6] pointed out that the metadata catalog is a key component of data lakes that prevents them from being data "swamps."

3 DATA LAKES ARCHITECTURES

Data Lake, as defined in the IBM Redbook, is a set of centralized repositories containing vast amounts of raw data (either structured or unstructured), described by metadata, organized into identifiable datasets, and available on demand. Figure 1 shows a standard proposal for an architecture of a data lake system [9]. The system consists of four layers which are 1) Ingestion Layer, 2) Storage Layer, 3) Transformation Layer, and 4) Interaction Layer.

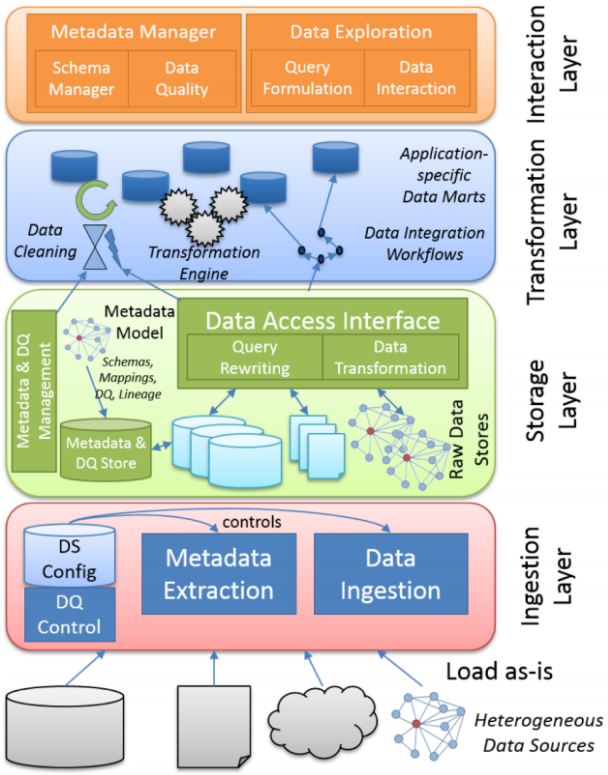

Figure 1: Data Lake Architecture

3.1 Ingestion Layer

The Ingestion Layer is in charge of bringing data into the data lake system from various sources. One of the main features of the data lake concept is the ease with which any type of data can be ingested and loaded. Data lakes, on the other hand, have been repeatedly reported as requiring governance in order to avoid becoming data swamps. Administrators of data lakes are in charge of this critical topic.

The metadata extractor is the most key component of the ingestion layer. It should make it easier for data lake administrators to configure new data sources and make them accessible in the data lake. To accomplish this, the metadata extractor should extract as much metadata as possible from the data source (for example, schemas from relational or XML sources) and store it in the data lake's metadata storage. The raw data, in addition to the metadata, must be absorbed into the data lake. As noted in [9], since the raw data are kept in their original format, this is more like a "copy" operation, which is certainly less complex than an ETL (Extract-Transform-Load) process in data warehouses.

3.2 Storage Layer

The metadata repository (discussed in Section 4) and the raw data repositories are the two key components of the storage layer. Since raw data repositories must be stored in their native format, data lake environments must support a variety of storage systems for relational, graph, XML, and JSON data. Hadoop appears to be a strong candidate for the storage layer's basic platform. To support data fidelity, however, additional components such as Tez or Falcon are needed.

3.3 Transformation Layer

Data can be transformed from storage to user experiences using the transformation layer. It requires operations such as cleansing, format transformations, and so on.

3.4 Interaction Layer

All of these functionalities that are needed to work with the data should be covered by the interaction layer. As mentioned in [9], these functionalities should include visualization, annotation, selection and filtering of data, as well as basic analytical methods. It's also worth noting that more advanced analytics like machine learning and data mining should not be regarded as part of a data lake system's capabilities.

4 METADATA

Metadata is a crucial concept for ensuring that information can survive and be accessible in the future. It specifies requirements for defining an information resource without ambiguity. It can also be used to organise information resources by creating connections between them based on how they are used or what they are about. It also contribute to defining open systems in which interoperability and integration of data sources are eased by formats like JSON, XML, and RDF.

There are about 16 different subject areas that might be chosen for inclusion in an enterprise metadata repository. Here are some of them

- **Data structure metadata:** involves data about physical files, relational database tables and columns, indexes and other database control objects.
- **Data modeling metadata:** involves data about business entities, their attributes, relationships between entities and business rules governing these relationships.
- **Integration metadata:** involves data about the mappings and transformations rules to migrate data from a source to a destination.
- **Business intelligence metadata:** involves data about business intelligence interfaces, queries, reports and usage.
- **Data security metadata:** involves data about users, privileges, and groups.
- **Content management metadata:** involves metadata about unstructured data found in documents, including taxonomies, ontologies, and search engine keywords.
- **Legacy system metadata:** involves data supporting impact analysis, restructuring, reuse and componentization of applications modules.

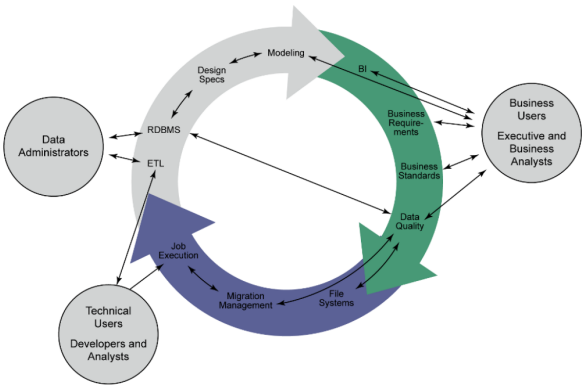

Figure 2: Point-to-Point Metadata Architecture

- **Application development metadata:** involves data about application business and technical requirements, application design, architecture, test plans, and test data.

The metadata that connects these subject areas, defining the relationships between data, processes, applications, and technology, may be the most valuable to the enterprise. This data may not be stored in any standard metadata source tool, but instead must be collected separately, perhaps by subject matter experts directly entering data into the integrated metadata repository.

4.1 Metadata Architecture

There are several metadata architecture approaches illustrated here.

4.1.1 Point-to-point metadata architecture. The point-to-point metadata architecture has an independent metadata repository for each software application. For example, as shown in Figure 2, a data modeling software creates and maintains entity/attribute name, definition, etc. This metadata repository will be accessed and updated by data modeling software. Similarly, BI applications contain metadata about reports, calculations and data derivations and consumers of data, and data integration software contains metadata on lineage.

Although this architecture is simple for implementation, it has some limitations.

Limitations:

- Difficulty involved in integrating metadata, to support complex searches and consolidate metadata views.
- Limited support for resolving semantic differences across tools.
- Does not support automated impact analysis.

4.1.2 Hub and Spoke Metadata Architecture. The hub and spoke metadata architecture, as shown in Figure 3 consists of a centralized repository that stores all metadata from each tool that creates metadata. The central repository is the hub, and the spokes are applications such as data modeling tool, BI applications and data integration software.

Although this architecture is ideal for metadata management as it handles the limitation of the previous architecture, it rarely

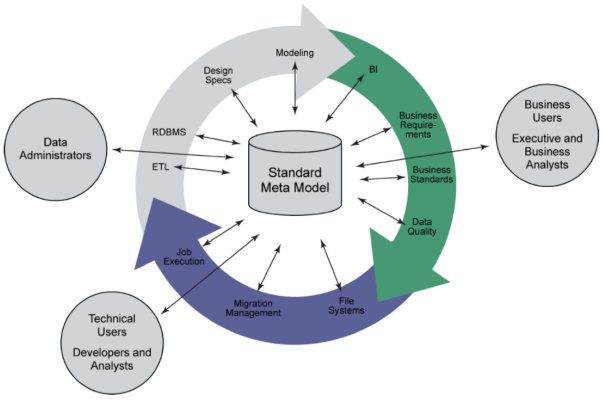

Figure 3: Hub and Spoke Metadata Architecture.

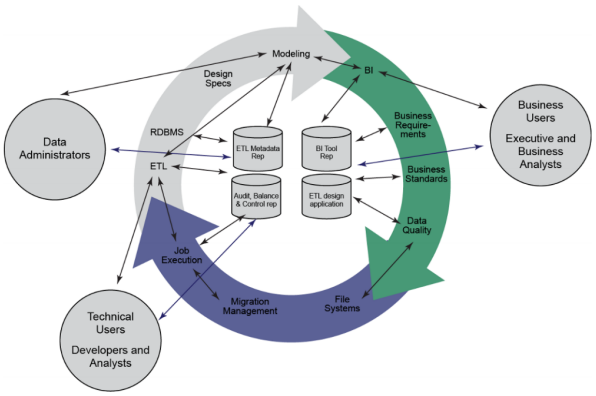

Figure 5: Hybrid Metadata Architecture

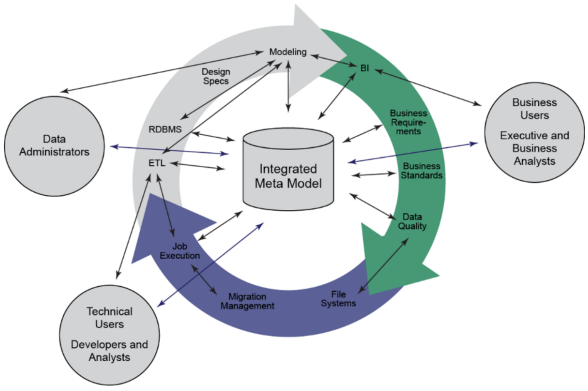

Figure 4: Tool of Record Metadata Architecture

attained due to the following limitations.

Limitations:

- Cost and complexity of implementation.
- It relies on a repository tool with extensive metadata exchange bridges.
- It requires extensive customization, requiring several years of effort and cost.

4.1.3 Tool of Record Metadata Architecture. It also uses a centralized repository, but this centralized repository stores only unique metadata from metadata sources of record as shown in Figure 4. The tool of record centralized repository is like building a data warehouse for metadata.

Although this architecture has less complexity to implement because it only collects unique metadata, not all., it still has some limitations.

Limitations:

- It relies on customization.
- It faces the same challenges of any data warehouse implementation (e.g. semantic and granularity issue resolution before metadata loading).

4.1.4 Hybrid Metadata Architecture. The hybrid metadata architecture uses some of the architectural highlights from the other architectures. A few tools are selected as metadata repositories or registries. Point-to-Point interfaces are created from metadata sources to these repositories. The hybrid metadata architecture removes some complexity and expense involved in implementing a centralized repository.

Although this architecture limits the number of repository-to-repository interfaces and supports highly customized presentation layers, advanced searches and partial automation of impact analysis across tools, the metadata answers are not all in a single location (**Limitation**).

4.1.5 Federated Metadata Architecture. The federated repository can be considered a virtual enterprise metadata repository. An architecture for the federated repository would look something like Figure 6. The bottom of the diagram is a series of separate sources of metadata and other metadata repositories, some located on premise and some located in the cloud. Virtualization is achieved through a series of connectors, with each connector designed to access the metadata held within the source system or repository. These connectors provide a means of abstraction to the integrated metadata store. This system comprises of automated services to ingest new metadata or analyze new data stores as they are added to the overall integrated metadata system. Finally, a single-user experience allows both business and IT users to find, access and view metadata. The portal provides different search methods such as SQL, free text and graph, in order to allow different communities to search for the metadata they are most interested in. The portal also provides collaboration to allow users to tag or provide additional descriptions and commentary on the metadata.

Although this architecture focuses on automated curation and providing a single view of metadata to the user and leverages an organization's current tools, it still has some limitations.

Limitations:

- It is not ideally suited for manual metadata curation.
- Limited market tool set to provide the virtualization.
- Architectural approach and software still maturing.

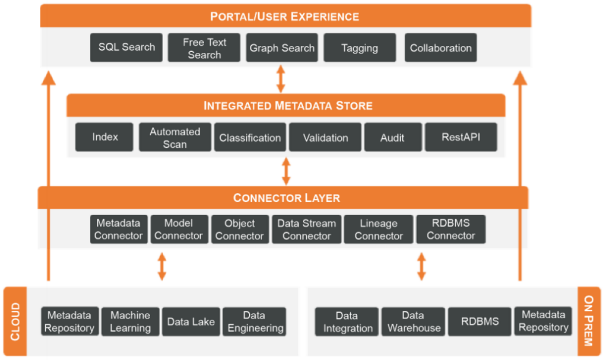

Figure 6: Federated Metadata Architecture

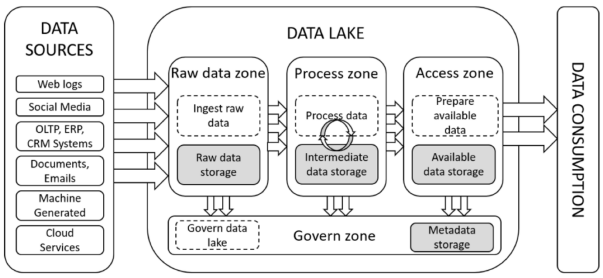

Figure 7: Functional Data Lake Architecture

4.2 Metadata Management in Data Lakes

Metadata are data and thus, metadata in data lakes have to be properly managed. In this section, we address this issue by giving hints on how to organize their storage, how to store them, how to discover them, how to define their lineage, how to query them and how to select the related data sources.

Metadata directory: The metadata directory gathers all metadata in a knowledge base whose structure is defined by a "metadata schema". In the knowledge base, metadata are organized according to a catalog. This metadata catalog is built up by experts of the application domain, independently from the processes for data processing. The goal of this catalog is to provide the necessary knowledge to properly monitor all services offered by the data ecosystem. This catalog can thus be seen as a reference dictionary for the identification and control of the data stored in the data lake.

Metadata storage: The storage of the data and their associated metadata is done during the initial loading phase, as well as during every updating phase of the ecosystem.

Metadata discovery: This process is for detecting implicit metadata that have not been extracted during the initial loading phase.

Metadata lineage: The lineage on data describes the transformations, the different states, the characteristics and the quality of these data during the processing chain.

Metadata querying: When querying metadata, the expected answer is the set of data associated in the data lake to the metadata specified by the query. These queries are generally complex queries expressed in various languages, based either on natural language or on SQL.

Data source selection: The choice of whether a new data source should be integrated relies on information such as the estimated cost of data importation or the estimated coverage of values by the dataset and quality of the data.

5 FUNCTIONAL ARCHITECTURE

To be as complete as possible and to answer the requirements of Toulouse UHC, the book authors propose a definition that includes input, process, output and governance of data lakes:

Data Lake: A data lake is a Big Data analytics solution that ingests heterogeneously structured raw data from various sources (local or external to the organization) and stores these raw data in their native format, allows us to process data according to different requirements and provides access of available data to different users (data scientists, data analysts, BI professionals, etc.) for statistical analysis, Business Intelligence (BI), Machine Learning (ML), etc., and governs data to ensure the data quality, data security and data lifecycle.

Data lake functional architecture has evolved from mono-zone [1, 4] to five-data-ponds [5] then to multi-zone [8], and it is always presented with technical solutions. None of the existing Data Lake architectures draw a clear distinction between functionality-related and technology-related components. What is more, the concept of multi-zone architecture is interesting and deserves further investigation. Some zones are required, while others are optional or reorganizable. In terms of the critical zones, a data lake should be able to ingest raw data, process data as required, store processed data, provide access for various uses, and regulate data, according to the data lake specification.

Functional architecture concerns the usage perspective and it can be implemented by different technical solutions. By adopting the existing data lake architectures and avoiding their shortcomings, we propose a functional data lake architecture for the project discussed in the book (Toulouse UHC) in Figure 7, which contains four essential zones, with each having a treatment area (dotted rectangle) and a data storage area that stores the result of processes (gray rectangle). The metadata classification for this functional archetcture is shown in Figure 8.

Raw Data Zone: all types of data are ingested without processing and are stored in their native format. The ingestion can be batch, real-time or hybrid. This zone allows users to find the original version of data for their analysis to facilitate subsequent treatments.

Process Zone: in this zone, users can transform data according to their requirements and store all the intermediate transformations. The data processing includes batch and/or real-time processing. This zone allows users to process data (selection, projection, join, aggregation, normalization etc.) for their data analysis.

Access Zone: users can put all the prepared data in the access zone which stores all the available data and provides data access. This zone allows users to access data for self-service data consumption for different analytics (reporting, statistical analysis, business intelligence analysis, machine learning algorithms).

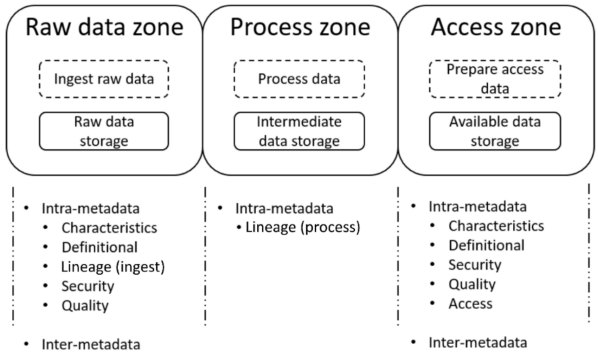

Figure 8: Meta Data Classification

5.1 Metadata Implementation

Metadata can be associated with data resources in many ways. There are three ways to do the association: (1) embedded metadata concerns the metadata integrated into the resource by authors, for example, XML documents; (2) associated metadata is maintained in files which are tightly linked to the resources; (3) third-party metadata is maintained in an individual repository. To store the metadata, a relational database is a good solution. This is the most used data storage in the organization. But NoSQL (as Graph DB) have more flexibility in the system; this way, the schema of metadata can be changed in the future to adapt the project.

5.1.1 Relational DBMS vs. Graph DBMS. Table 1 summarizes the differences between the two methods of implementation.

	Relational DBMS	**Graph DBMS**
Scalability	Vertical scalability (volume growing with high cost)	Horizontal scalability (volume with low cost)
Flexibility	Modify database schema with lots of effort	Modify database schema with less effort
Query	Standard language. Complicated when there are many joins	Various languages. Easy to find related nodes
Security	Mature	Security Mature Mature in some graph DBMSs

Table 1: Comparison of Relational and Graph DBMS

5.1.2 Master Data and the Data Lake. Another important issue that must be handled during the Metadata management process, is differentiate the Master Data from the Reference Data and managing these data.

Master data: according to the Gartner definition1: Master data management is a technology-enabled discipline in which business and IT work together to ensure the uniformity, accuracy, stewardship, semantic consistency and accountability of the enterprise official shared master data assets. Master data is the consistent and uniform set of identifiers and extended attributes that describes the core entities of the enterprise including customers, prospects, citizens, suppliers, sites, hierarchies and chart of accounts.

Reference data: reference data refers to the data residing in code tables or lookup tables. They are normally static code tables storing values such as city and state codes, zip codes, product codes, country codes and industry classification codes. Reference data has general characteristics. They are typically used in a read-only manner by operational, analytical and definitional systems. Reference data can be defined internally or specified externally by standards bodies groups (ISO, ANSI, etc.). Reference data can have a taxonomy, for example, hierarchy

There are two ways to master data in a data lake: 1) feeding mastered data into the lake from the MDM hub; 2) mastering data in the data lake itself. In the first approach, companies use an MDM hub to master the data. The MDM hub improves the quality of core data that is fed into the data lake. In the second approach, companies that have an extraordinarily high number of records can master the data within the data lake itself. This frees up data scientists to spend more time exploring and analyzing, and less time trying to fix data issues, such as duplicate customer records. It also helps data scientists understand the relationships between the data.

6 LINKED DATA

Linked Data can play an important role in Data Lakes as it can overcome traditional databases because it has a way of naturally establishing strong relationships. In linked data, data are stored in their original source form, and are therefore made readily available for querying in such a form. This may be regarded as an advantage because a data lake structured in this way can be generated fairly quickly with minimum transformation overhead.

One criticism against data lakes is that, in order to be exploitable, a data lake requires or assumes a degree of knowledge on the consumers' part of the context and processes behind the ingestion of those data, and this information is likely to be lost or not recorded at data generation time. The Linked Data principles have a way of mitigating this drawback. For one thing, while there exists a notion of dataset in Linked Data, it is a very loose one, in that the granularity of a dataset may be associated to that of an RDF graph or a set of one or more graphs that can be accessed through the same service endpoint for querying (in the SPARQL language). A consequence of this is that, given a mass of linked data, it is possible to logically partition it into datasets without fundamentally altering the structure and semantics of its content.

The downside of this approach is that, with the rules for integration being hard-coded in the query for retrieving the data, the burden of managing these rules still falls on the final consumer of the data, despite it being fairly easy to generate rules that approximate precise data integration. There are also software systems that, given a canonical non-federated SPARQL query, attempt to fetch results from multiple datasets that are able to satisfy at least some part of the query. This is generally not transparent, meaning that the user does not need to specify, or does not even need to be aware of, which datasets to query for which patterns in the original query

REFERENCES

[1] Huang Fang. 2015. Managing data lakes in big data era: What's a data lake and why has it became popular in data management ecosystem. In *2015 IEEE International*

Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER). IEEE, 820–824.

[2] GARTNER. 2014. Gartner says beware of the data lake fallacy. http://www.gartner.com/newsroom/id/2809117.

[3] Bill Inmon. 2016. *Data Lake Architecture: Designing the Data Lake and avoiding the garbage dump.* Technics publications.

[4] DIXON J. 2010. Pentaho, Hadoop, and Data Lake. https://jamesdixon.wordpress.com/2010/10/14/pentaho-hadoop-and-data-lakes/.

[5] Cedrine Madera and Anne Laurent. 2016. The next information architecture evolution: the data lake wave. In *Proceedings of the 8th International Conference on Management of Digital EcoSystems*. 174–180.

[6] MARKETSANDMARKETS. 2014. Governing and managing big data for analytics and decision makers. http://www.redbooks.ibm.com/abstracts/redp5120.html?Open.

[7] MARKETSANDMARKETS. 2016. Data lakes market by software. https://www.marketsandmarkets.com/Market-Reports/data-lakes-market.

[8] Rajesh Nadipalli. 2017. *Effective Business Intelligence with QuickSight.* Packt Publishing Ltd.

[9] Sherif Sakr and Albert Y Zomaya. 2019. *Encyclopedia of big data technologies*. Springer International Publishing.